

Retail Store Sales Analysis

1. Project Introduction

Retail stores generate a large amount of sales data every day. This data contains information about customers, products, sales, profit, payment methods, and regions. By analyzing this data, businesses can understand their performance, identify trends, and make better business decisions. In this project, we use Python to clean, analyze, and visualize retail sales data. The analysis helps identify top-selling products, profitable regions, customer purchasing patterns, and overall sales performance.

2. Problem Statement

Retail stores generate a large amount of sales data. It is difficult to understand business performance from raw data. This project helps convert raw data into meaningful insights.

3. Objective of the Project

- Understand the dataset.
- Clean the data.
- Analyze sales performance.
- Create charts and graphs.
- Find business insights.

4.Data Set Description

This dataset contains retail store sales data.It includes information such as order details, customer information, product category, sales, profit, payment method, and shipping mode. This data is used to analyze sales performance and generate business insights.

5. Tools and Libraries Used

Libraries

- Pandas – Data loading, cleaning, and analysis
- NumPy – Numerical calculations
- Matplotlib – Data visualization
- Seaborn – Statistical charts

Development Environment

- Jupyter Notebook
- VS Code

STEP 1: Import Libraries

In this step import the required Python libraries for data loading, analysis, and visualization.

```
In [1]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
```

STEP 2: Load Dataset

In this step Load the retail sales dataset into a Pandas DataFrame.

```
In [2]: df=pd.read_excel("Retail_Store_Sales_Raw_Data.xlsx")
```

STEP 3: Data Understanding

In this step, we explore the dataset to understand its structure, columns, data types, missing values, and basic statistics. This helps us identify data quality issues before performing data cleaning and analysis.

```
In [3]: df.head()
```

```
Out[3]:
```

	OrdID	Order_Date	Cust_ID	Customer_Name	City	Region	Category	P
0	ORD10001	45558	C5065	Aditya Sharma	Delhi	north	Office Supplies	
1	ORD10002	45615	C5484	Neha Sharma	Hyderabad	South	Office Supplies	
2	ORD10003	45301	C5085	Rohan Jadhav	Ahmedabad	West	Electronics	
3	ORD10004	45658	C5405	Sanket Naik	Delhi	North	Office Supplies	
4	ORD10005	45594	C5346	Vishal Kadam	Mumbai	West	Electronics	

```
In [4]: df.tail()
```

Out[4]:

	OrdID	Order_Date	Cust_ID	Customer_Name	City	Region	Category
1498	ORD11499	45686	C5334	Komal Sharma	Delhi	North	Office Supplies
1499	ORD11500	45666	C5103	Priya Naik	Hyderabad	South	Electronics
1500	ORD11444	45696	C5272	Sanket Verma	Ahmedabad	West	Electronics
1501	ORD11445	45390	C5336	Aditya Singh	Bangalore	South	Electronics
1502	ORD11389	45640	C5386	Kiran Pawar	Kolkata	East	Office Supplies

In [5]:

```
df.sample(5)
```

Out[5]:

	OrdID	Order_Date	Cust_ID	Customer_Name	City	Region	Category
790	ORD10791	45746	C5277	Rahul Desai	Pune	West	Furniture
488	ORD10489	45462	C5049	Sanket Joshi	Ahmedabad	West	Office Supplies
1024	ORD11025	45815	C5374	Pooja Jadhav	Ahmedabad	West	Electronics
90	ORD10091	45512	C5479	Akash Pawar	Delhi	North	Electronics
1029	ORD11030	45514	C5188	Rahul Verma	Ahmedabad	West	Electronics

In [6]:

```
row,column= df.shape
print("row:",row)
print("column:",column)
```

row: 1503
column: 15

In [7]:

```
print(df.columns.tolist())
```

```
['OrdID', 'Order_Date', 'Cust_ID', 'Customer_Name', 'City', 'Region', 'Category',  
'Product', 'Quantity', 'Unit_Price', 'Discount_%', 'Sales', 'Profit', 'Payment_Me  
thod', 'Ship_Mode']
```

In [8]:

```
df.index
```

Out[8]: RangeIndex(start=0, stop=1503, step=1)

In [9]:

```
df.dtypes
```

```
Out[9]: OrdID          str
        Order_Date    int64
        Cust_ID       str
        Customer_Name  str
        City          str
        Region        str
        Category      str
        Product       str
        Quantity      int64
        Unit_Price    float64
        Discount_%    float64
        Sales         float64
        Profit        float64
        Payment_Method str
        Ship_Mode     str
        dtype: object
```

```
In [10]: df.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 1503 entries, 0 to 1502
Data columns (total 15 columns):
 #   Column          Non-Null Count  Dtype
---  -
 0   OrdID           1503 non-null  str
 1   Order_Date      1503 non-null  int64
 2   Cust_ID         1503 non-null  str
 3   Customer_Name   1503 non-null  str
 4   City            1503 non-null  str
 5   Region          1503 non-null  str
 6   Category        1503 non-null  str
 7   Product         1503 non-null  str
 8   Quantity        1503 non-null  int64
 9   Unit_Price      1503 non-null  float64
10  Discount_%      1187 non-null  float64
11  Sales           1503 non-null  float64
12  Profit          1503 non-null  float64
13  Payment_Method  1503 non-null  str
14  Ship_Mode       1503 non-null  str
dtypes: float64(4), int64(2), str(9)
memory usage: 176.3 KB
```

```
In [11]: df.describe()
```

Out[11]:

	Order_Date	Quantity	Unit_Price	Discount_%	Sales	Pr
count	1503.000000	1503.000000	1503.000000	1187.000000	1503.000000	1503.000
mean	45558.081836	4.603460	25524.683806	12.540017	105121.359281	17651.666
std	157.281747	2.281994	15525.108442	5.683980	84902.131697	16605.481
min	45292.000000	1.000000	159.500000	5.000000	198.070000	24.810
25%	45416.500000	3.000000	13139.275000	5.000000	34814.730000	5201.810
50%	45562.000000	5.000000	25281.000000	10.000000	84234.360000	12466.730
75%	45692.000000	7.000000	37859.280000	20.000000	158648.140000	25229.900
max	45832.000000	8.000000	249257.090000	20.000000	387319.840000	96814.770



STEP 4: Data Cleaning

In this step, we clean the dataset by handling missing values, removing duplicate records, correcting data types, fixing inconsistent values, and preparing the data for accurate analysis. This improves the quality and reliability of the dataset.

4.1] Missing Values

In [12]: `df.isnull()`

Out[12]:

	OrdID	Order_Date	Cust_ID	Customer_Name	City	Region	Category	Product
0	False	False	False	False	False	False	False	False
1	False	False	False	False	False	False	False	False
2	False	False	False	False	False	False	False	False
3	False	False	False	False	False	False	False	False
4	False	False	False	False	False	False	False	False
...	...	...	...	...	...	...	...	...
1498	False	False	False	False	False	False	False	False
1499	False	False	False	False	False	False	False	False
1500	False	False	False	False	False	False	False	False
1501	False	False	False	False	False	False	False	False
1502	False	False	False	False	False	False	False	False

1503 rows × 15 columns

In [13]: `df.isnull().sum()`

```
Out[13]: OrdID          0
         Order_Date     0
         Cust_ID        0
         Customer_Name   0
         City           0
         Region         0
         Category       0
         Product        0
         Quantity       0
         Unit_Price     0
         Discount_%     316
         Sales          0
         Profit         0
         Payment_Method  0
         Ship_Mode      0
         dtype: int64
```

```
In [14]: df["Discount_%"] = df["Discount_%"].fillna(0)
```

```
In [15]: df.isnull().sum()
```

```
Out[15]: OrdID          0
         Order_Date     0
         Cust_ID        0
         Customer_Name   0
         City           0
         Region         0
         Category       0
         Product        0
         Quantity       0
         Unit_Price     0
         Discount_%     0
         Sales          0
         Profit         0
         Payment_Method  0
         Ship_Mode      0
         dtype: int64
```

4.2] Duplicate Records

```
In [16]: df.duplicated()
```

```
Out[16]: 0      False
         1      False
         2      False
         3      False
         4      False
         ...
        1498    False
        1499    False
        1500     True
        1501     True
        1502     True
        Length: 1503, dtype: bool
```

```
In [17]: df.duplicated().sum()
```

```
Out[17]: np.int64(3)
```

```
In [18]: df=df.drop_duplicates()
```

```
In [19]: df.duplicated().sum()
```

```
Out[19]: np.int64(0)
```

4.3] Incorrect DataType

```
In [20]: print(df.dtypes)
```

```
OrdID          str
Order_Date     int64
Cust_ID        str
Customer_Name  str
City           str
Region         str
Category       str
Product        str
Quantity       int64
Unit_Price     float64
Discount_%     float64
Sales          float64
Profit         float64
Payment_Method str
Ship_Mode      str
dtype: object
```

```
In [21]: df["Order_Date"]=pd.to_datetime(df["Order_Date"])
```

```
In [22]: df.dtypes
```

```
Out[22]: OrdID          str
Order_Date     datetime64[ns]
Cust_ID        str
Customer_Name  str
City           str
Region         str
Category       str
Product        str
Quantity       int64
Unit_Price     float64
Discount_%     float64
Sales          float64
Profit         float64
Payment_Method str
Ship_Mode      str
dtype: object
```

4.4] Replace Incorrect Values

```
In [23]: print("City      : ",df["City"].unique().tolist())
print("Region   : ",df["Region"].unique().tolist())
print("Category: ",df["Category"].unique().tolist())
print("Product  : ",df["Product"].unique().tolist())
print("Payment_Method: ",df["Payment_Method"].unique().tolist())
print("Ship_Mode: ",df["Ship_Mode"].unique().tolist())
```

```

City    : ['Delhi', 'Hyderabad', 'Ahmedabad', 'Delhi', 'Mumbai', 'Kolkata', 'Pune', 'Pune', 'Chennai', 'Hyderabad', 'pune', 'Chennai', 'Bangalore', 'Dalhi']
Region  : ['north', 'South', 'West', 'North', 'East']
Category: ['Office Supplies', 'Electronics', 'Furniture']
Product : ['Marker', 'Laptop', 'Paper', 'Table', 'Keyboard', 'Headphones', 'Pen', 'Notebook', 'Desk', 'Chair', 'File Folder', 'Monitor', 'Sofa', 'Bookshelf', 'Mobile']
Payment_Method: ['UPI', 'Cash', 'Credit Card', 'Debit Card', 'upi', 'cash']
Ship_Mode: ['Standard', 'Express', 'Same Day']

```

```

In [24]: #Remove Extra Space
df["City"]=df["City"].str.strip()

```

```

In [25]: print("City    :",df["City"].unique().tolist())
print("Region   :",df["Region"].unique().tolist())
print("Payment_Method:",df["Payment_Method"].unique().tolist())

```

```

City    : ['Delhi', 'Hyderabad', 'Ahmedabad', 'Mumbai', 'Kolkata', 'Pune', 'Chennai', 'pune', 'Bangalore', 'Dalhi']
Region  : ['north', 'South', 'West', 'North', 'East']
Payment_Method: ['UPI', 'Cash', 'Credit Card', 'Debit Card', 'upi', 'cash']

```

```

In [26]: df["City"]=df["City"].replace({
        "pune":"Pune",
        "Dalhi":"Delhi"
    })

df["Region"]=df["Region"].replace({
    "north":"North"
})

df["Payment_Method"]=df["Payment_Method"].replace({
    "upi":"UPI", "cash":"Cash"
})

```

```

In [27]: print("City    :",df["City"].unique().tolist())
print("Region   :",df["Region"].unique().tolist())
print("Payment_Method:",df["Payment_Method"].unique().tolist())

```

```

City    : ['Delhi', 'Hyderabad', 'Ahmedabad', 'Mumbai', 'Kolkata', 'Pune', 'Chennai', 'Bangalore']
Region  : ['North', 'South', 'West', 'East']
Payment_Method: ['UPI', 'Cash', 'Credit Card', 'Debit Card']

```

4.5] Rename Columns

```

In [28]: df.columns

```

```

Out[28]: Index(['OrdID', 'Order_Date', 'Cust_ID', 'Customer_Name', 'City', 'Region',
               'Category', 'Product', 'Quantity', 'Unit_Price', 'Discount_%', 'Sales',
               'Profit', 'Payment_Method', 'Ship_Mode'],
              dtype='str')

```

```

In [29]: df.rename(columns={"OrdID":"Order_ID", "Cust_ID":"Customer_ID"}, inplace=True)

```

```

In [ ]:

```

```

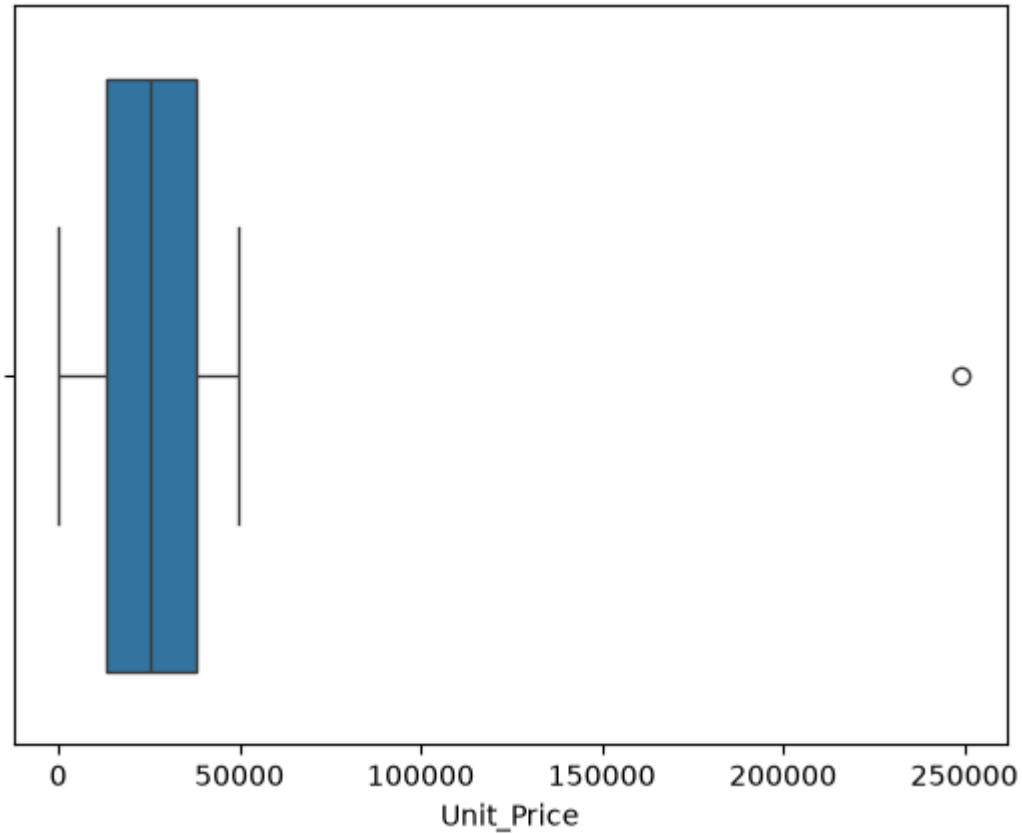
In [30]: df.columns

```

```
Out[30]: Index(['Order_ID', 'Order_Date', 'Customer_ID', 'Customer_Name', 'City',  
              'Region', 'Category', 'Product', 'Quantity', 'Unit_Price', 'Discount_%',  
              'Sales', 'Profit', 'Payment_Method', 'Ship_Mode'],  
             dtype='str')
```

4.6] Outlier Detection

```
In [31]: sns.boxplot(x=df["Unit_Price"])  
plt.show()
```



STEP 5: Verify Cleaned Data

in this step confirms the dataset is proper cleaned

```
In [32]: df.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 1500 entries, 0 to 1499
Data columns (total 15 columns):
 #   Column                Non-Null Count  Dtype
---  -
 0   Order_ID              1500 non-null   str
 1   Order_Date            1500 non-null   datetime64[ns]
 2   Customer_ID           1500 non-null   str
 3   Customer_Name         1500 non-null   str
 4   City                  1500 non-null   str
 5   Region                1500 non-null   str
 6   Category              1500 non-null   str
 7   Product               1500 non-null   str
 8   Quantity              1500 non-null   int64
 9   Unit_Price            1500 non-null   float64
10   Discount_%            1500 non-null   float64
11   Sales                 1500 non-null   float64
12   Profit                1500 non-null   float64
13   Payment_Method        1500 non-null   str
14   Ship_Mode             1500 non-null   str
dtypes: datetime64[ns](1), float64(4), int64(1), str(9)
memory usage: 175.9 KB
```

```
In [33]: df.isnull().sum().sum()
```

```
Out[33]: np.int64(0)
```

```
In [34]: df.duplicated().sum()
```

```
Out[34]: np.int64(0)
```

STEP 6: Export Cleaned Data

The cleaned data is saved further analysis and visualization

```
In [35]: df.to_excel("Retail_Sales_Clean.xlsx", index=False)
```

STEP 7: Reload Cleaned Data

The cleaned data is reload to ensure analysis and visualization are done on clean data

```
In [36]: df=pd.read_excel("Retail_Sales_Clean.xlsx")
```

STEP 8: Data Analysis

In this step, we analyze the cleaned dataset to identify sales trends, customer behavior, product performance, and business performance. The analysis helps convert raw data into meaningful insights for better decision-making.

8.1] Overall statistical summary

- Get a statistical summary of all numerical columns to understand the distribution of the data.

In [37]: `df.describe()`

Out[37]:

	Order_Date	Quantity	Unit_Price	Discount_%	Sales	Profit
count	1500	1500.000000	1500.000000	1500.000000	1500.000000	1500.000000
mean	1970-01-01 00:00:00	4.599333	25518.603673	9.900000	104993.369073	17654.315000
min	1970-01-01 00:00:00	1.000000	159.500000	0.000000	198.070000	24.810000
25%	1970-01-01 00:00:00	3.000000	13147.597500	5.000000	34820.130000	5208.690000
50%	1970-01-01 00:00:00	5.000000	25243.510000	10.000000	84187.410000	12410.170000
75%	1970-01-01 00:00:00	7.000000	37842.805000	15.000000	157622.897500	25228.395000
max	1970-01-01 00:00:00	8.000000	249257.090000	20.000000	387319.840000	96814.770000
std	NaN	2.282018	15520.954693	7.191974	84826.186178	16616.714700

8.2] Total Sales

Calculate the total sales generated by the store.

In [38]: `df["Sales"].sum()`

Out[38]: `np.float64(157490053.60999998)`

8.3] Total Profit

Calculate the total profit earned from all orders.

In [39]: `df["Profit"].sum()`

Out[39]: `np.float64(26481472.58)`

8.4] Average Sales

Find the average sales value per order.

In [40]: `df["Sales"].mean()`

Out[40]: `np.float64(104993.36907333332)`

8.5] Average Profit

Find the average profit earned per order.

In [41]: `df["Profit"].mean()`

```
Out[41]: np.float64(17654.315053333332)
```

8.6] Total Orders

Count the total number of orders in the dataset.

```
In [42]: df["Order_ID"].count()
```

```
Out[42]: np.int64(1500)
```

8.7] Business Analysis

1. Top 10 product by sales

```
In [43]: df.groupby("Product")["Sales"].sum().sort_values(ascending=False).head(10)
```

```
Out[43]: Product
Desk      12463832.56
Headphones 12391995.37
Mobile    12028838.92
Sofa      11967116.72
Table     11720017.14
Paper     11067743.23
Pen       11064718.66
Monitor   10372884.90
File Folder 10022219.02
Marker    9850068.20
Name: Sales, dtype: float64
```

2. Top 10 products by profit

```
In [44]: df.groupby("Product")["Profit"].sum().sort_values(ascending=False).head(10)
```

```
Out[44]: Product
Headphones 2219548.31
Sofa       2062938.49
Mobile     2035658.16
Desk       2029778.60
Pen        1937789.34
Monitor    1794748.42
Table      1794520.41
Marker     1735164.59
File Folder 1716970.34
Laptop     1679878.84
Name: Profit, dtype: float64
```

3. Sales by Category

```
In [45]: df.groupby("Category")["Sales"].sum().sort_values(ascending=False)
```

```
Out[45]: Category
Furniture      53125593.68
Electronics    53092665.76
Office Supplies 51271794.17
Name: Sales, dtype: float64
```

4. Profit by Category

```
In [46]: df.groupby("Category")["Profit"].sum().sort_values(ascending=False)
```

```
Out[46]: Category
Electronics      9239414.37
Office Supplies  8661350.98
Furniture        8580707.23
Name: Profit, dtype: float64
```

5. Sales by Region

```
In [47]: df.groupby("Region")["Sales"].sum().sort_values(ascending=False)
```

```
Out[47]: Region
West      61046726.68
South     53542883.85
North     21503271.58
East      21397171.50
Name: Sales, dtype: float64
```

6. Profit by Region

```
In [48]: df.groupby("Region")["Profit"].sum().sort_values(ascending=False)
```

```
Out[48]: Region
West      10174208.06
South     8835266.71
North     3761779.66
East      3710218.15
Name: Profit, dtype: float64
```

7. Sales by City

```
In [49]: df.groupby("City")["Sales"].sum().sort_values(ascending=False)
```

```
Out[49]: City
Mumbai      22940225.15
Delhi       21503271.58
Kolkata     21397171.50
Ahmedabad   21174015.46
Bangalore   18350867.09
Chennai     17772345.54
Hyderabad   17419671.22
Pune        16932486.07
Name: Sales, dtype: float64
```

8. Payment Method Analysis

```
In [50]: df["Payment_Method"].value_counts()
```

```
Out[50]: Payment_Method
Cash      397
UPI       374
Credit Card 373
Debit Card 356
Name: count, dtype: int64
```

9. Shipping Mode Analysis

```
In [51]: df["Ship_Mode"].value_counts()
```

```
Out[51]: Ship_Mode
Standard      519
Express       493
Same Day      488
Name: count, dtype: int64
```

10. Monthly Sales Trend

```
In [52]: df.groupby(df["Order_Date"].dt.month)["Sales"].sum()
```

```
Out[52]: Order_Date
1      1.574901e+08
Name: Sales, dtype: float64
```

11. Top 10 Customers by Sales

```
In [53]: df.groupby("Customer_Name")["Sales"].sum().sort_values(ascending=False).head(10)
```

```
Out[53]: Customer_Name
Pooja Verma      2237192.21
Neha Kadam       2075264.85
Aditya Kadam     1869842.81
Neha Naik        1859712.95
Ravi Kadam       1850958.01
Rohan Patil      1831039.99
Ravi Jadhav      1820966.19
Sanket Kadam     1820824.14
Komal Pawar      1818746.73
Sneha Patil      1802016.41
Name: Sales, dtype: float64
```

12. Correlation Analysis

```
In [54]: df[["Quantity", "Unit_Price", "Sales", "Profit"]].corr()
```

```
Out[54]:
```

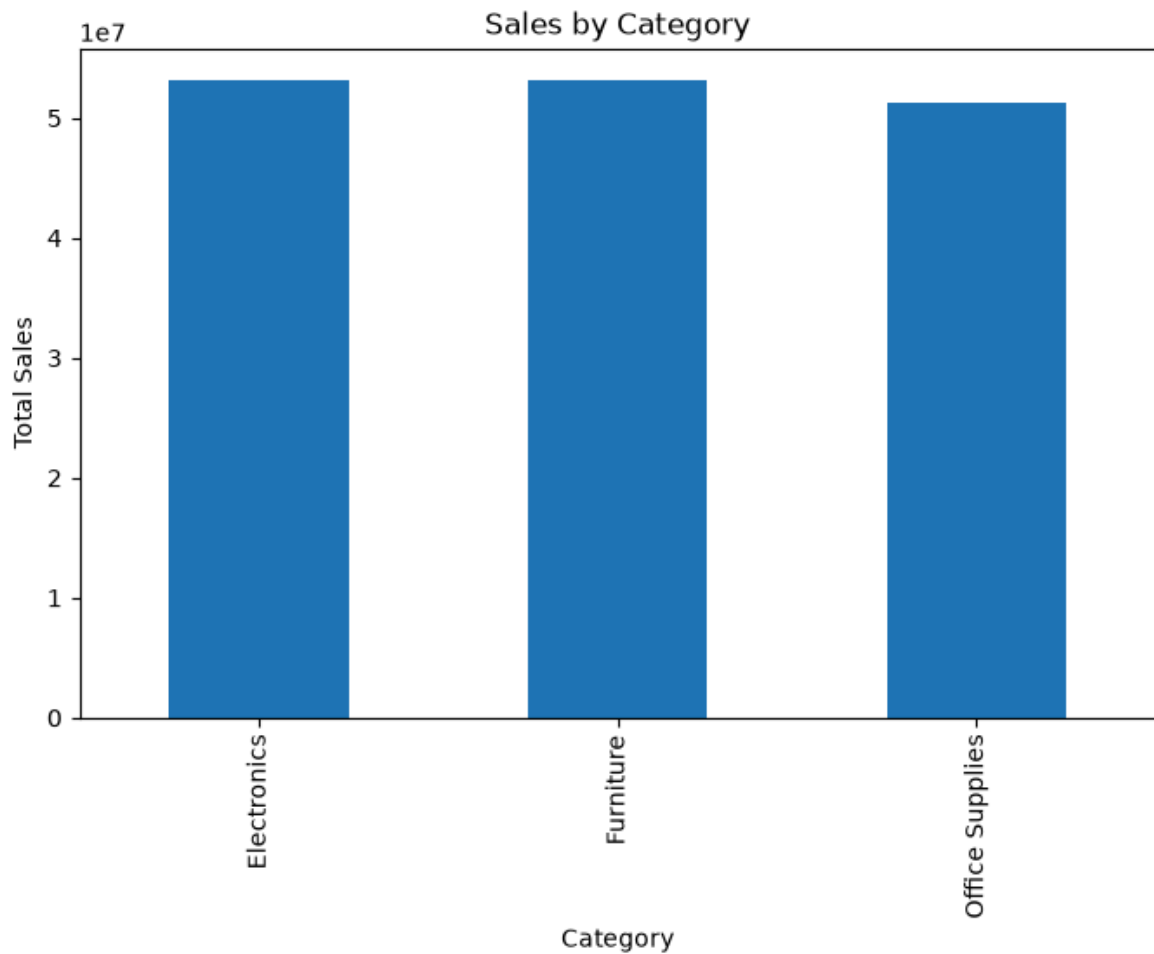
	Quantity	Unit_Price	Sales	Profit
Quantity	1.000000	0.002483	0.614166	0.518252
Unit_Price	0.002483	1.000000	0.668168	0.538439
Sales	0.614166	0.668168	1.000000	0.819238
Profit	0.518252	0.538439	0.819238	1.000000

STEP 9: Data Visualization

In this step, we create charts and graphs to present the analysis visually. Visualizations make it easier to identify trends, compare performance, and communicate business insights effectively.

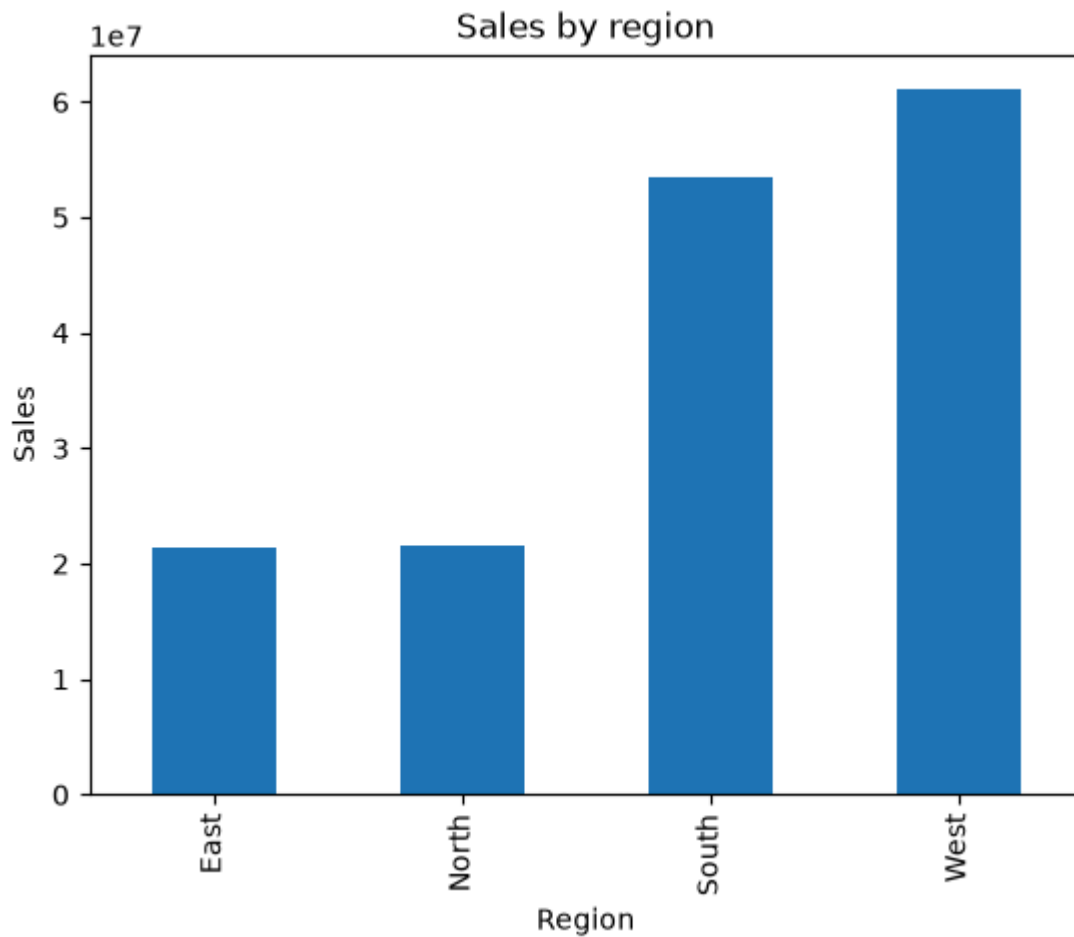
9.1) Sales by Category (Bar Chart)

```
In [55]: plt.figure(figsize=(8,5))
df.groupby("Category")["Sales"].sum().plot(kind="bar")
plt.title("Sales by Category")
plt.xlabel("Category")
plt.ylabel("Total Sales")
plt.show()
```



9.2) Sales by Region (Bar Chart)

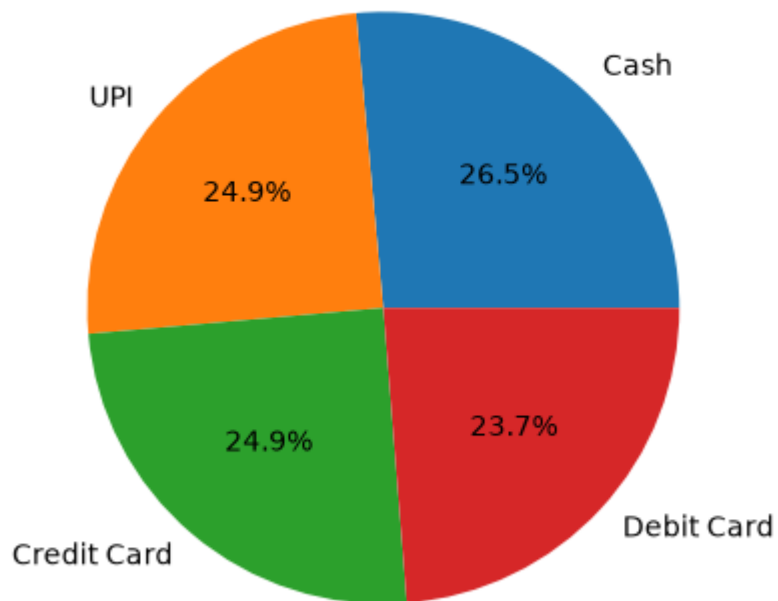
```
In [56]: df.groupby("Region")["Sales"].sum().plot(kind="bar")
plt.title("Sales by region")
plt.xlabel("Region")
plt.ylabel("Sales")
plt.show()
```



9.3) Payment Method Distribution (Pie Chart)

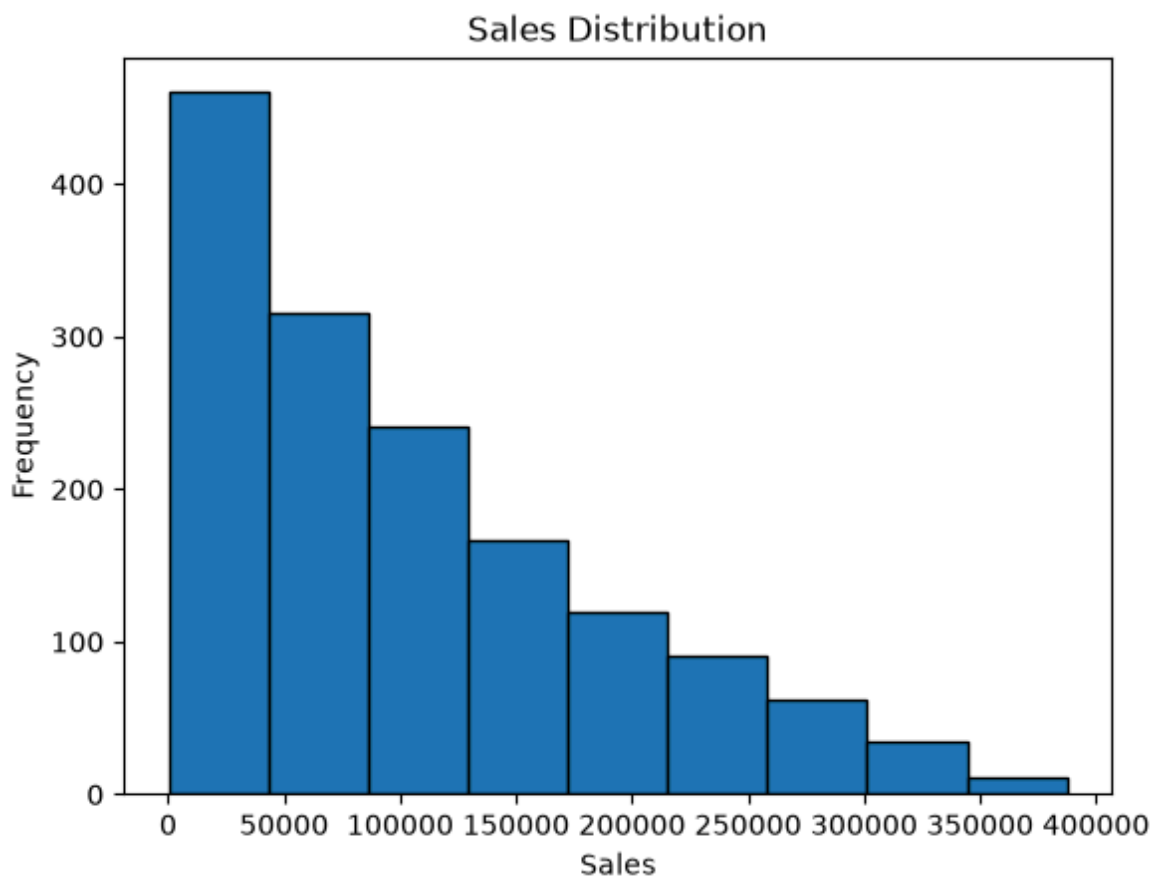
```
In [57]: df["Payment_Method"].value_counts().plot(
          kind="pie",
          autopct="%1.1f%%")
plt.title("Payment Method Distribution")
plt.show()
```

Payment Method Distribution



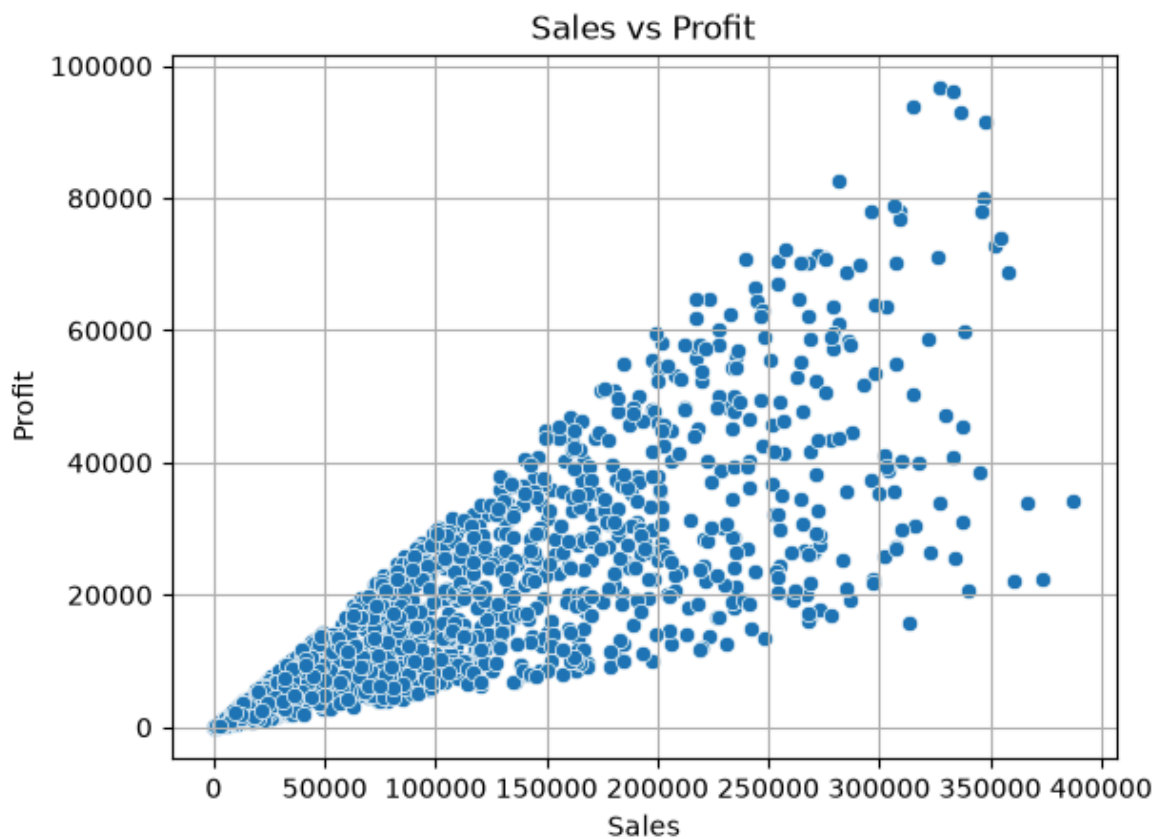
9.4) Sales Distribution (Histogram)

```
In [58]: plt.hist(df["Sales"],bins=9,edgecolor="Black")
plt.title("Sales Distribution")
plt.xlabel("Sales")
plt.ylabel("Frequency")
plt.show()
```



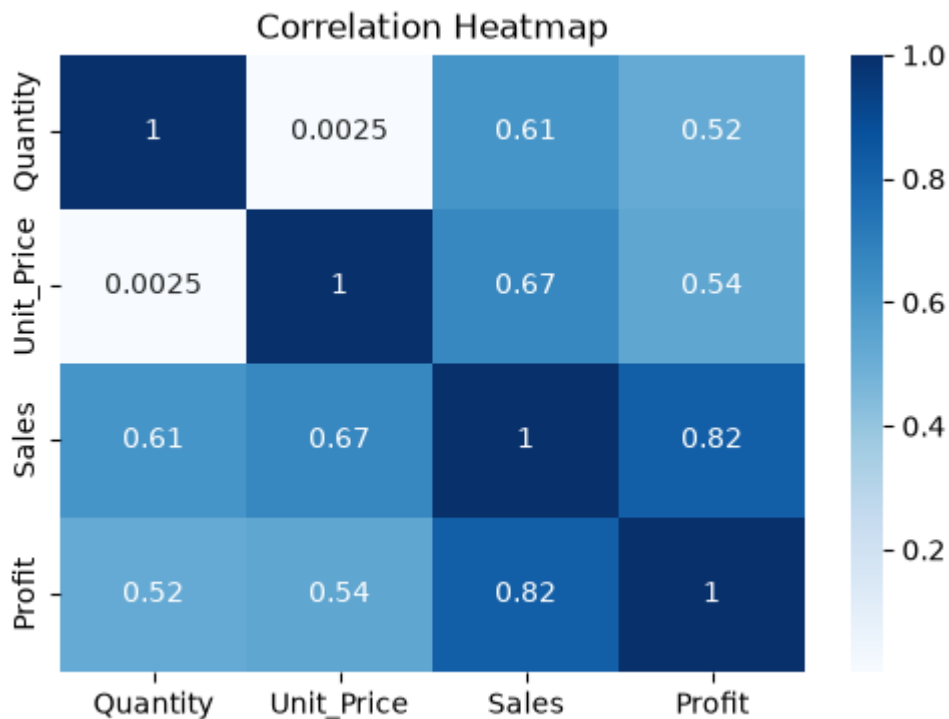
9.5) Sales vs Profit (Scatter Plot)

```
In [59]: sns.scatterplot(data=df, x="Sales",y="Profit")  
plt.title("Sales vs Profit")  
plt.grid()  
plt.show()
```



9.6) Correlation Heatmap

```
In [60]: plt.figure(figsize=(6,4))  
sns.heatmap(  
    df[["Quantity", "Unit_Price", "Sales", "Profit"]].corr(),  
    annot=True,  
    cmap="Blues"  
)  
plt.title("Correlation Heatmap")  
plt.show()
```



STEP 7: Business Insights

Business insights summarize the key findings from the analysis. They help decision-makers understand sales performance and make better business decisions.

1. The total sales generated by the store were **₹157,490,053.61**.
2. The total profit earned was **₹6,481,472.58**.
3. The average sales per order were **₹104,993.37**.
4. The average profit per order was **₹17,654.32**.
5. The dataset contains **1,500** orders.
6. **Desk** generated the highest sales among all products.
7. **Headphones** generated the highest profit among all products.
8. The **Furniture** category generated the highest sales.
9. The **Electronics** category generated the highest profit.
10. The **West** region recorded the highest sales and profit.
11. **Mumbai** generated the highest sales among all cities.
12. **Cash** was the most preferred payment method.
13. **Standard** Shipping was the most preferred shipping mode.

Step 9: Conclusion

The conclusion summarizes the overall outcome of the project.

Conclusion

This project analyzed retail store sales data using Python. The data was cleaned, explored, and visualized using various EDA techniques. The analysis identified important business trends, including top-performing products, categories, regions, cities, payment

methods, and shipping modes. Overall, the project demonstrates how Python can transform raw sales data into meaningful business insights that support better decision-making.